

ФИНАЛЬНАЯ МИССИЯ: КИБЕР-ОХОТНИК 2.0

МОДУЛЬ: СОСТОЯНИЕ, ЦИКЛЫ И ЭКОНОМИКА | ТЕОРИЯ

ЧТО ТАКОЕ СОСТОЯНИЕ (STATE)?

Представь, что ты играешь в игру. У тебя есть очки здоровья, количество монет и уровень персонажа. Эти значения могут меняться во время игры - это и есть состояние.

В программировании состояние хранится в переменных. Когда значение переменной меняется, мы хотим, чтобы экран сразу отобразил новое значение. В обычном программировании нам приходилось вручную обновлять каждый элемент на экране.

Jetpack Compose делает это автоматически! Мы используем специальную функцию `mutableStateOf()`, которая создаёт 'умную' переменную. Когда её значение меняется, Compose сам перерисовывает все элементы, которые используют эту переменную.

КАК РАБОТАЕТ `remember` И `mutableStateOf`?

`remember` - это функция, которая говорит Android: 'Запомни это значение и не создавай его заново при каждой перерисовке'. Без `remember` переменная бы сбрасывалась каждый раз.

`mutableStateOf()` создаёт отслеживаемое значение. Слово 'mutable' означает 'изменяемый', а 'state' - 'состояние'.

Синтаксис `'by remember { mutableStateOf(0) }'` позволяет нам работать с переменной как со обычной, но за кулисами Android следит за всеми изменениями.

Когда ты пишешь `credits += 10`, Compose автоматически находит все элементы экрана, где используется переменная `credits`, и обновляет только их.

ТИПЫ ДАННЫХ В KOTLIN

В Kotlin каждая переменная имеет тип. Основные типы для наших игр:

`Long` (числа с буквой L) - большие целые числа. Используем для денег, потому что игрок может накопить миллиарды!

`Int` - обычные целые числа. Подходит для уровней и количества предметов.

`Float` (числа с буквой f) - дробные числа. Нужно для процентов, прогресса и температуры.

`String` (в кавычках "") - текст. Имена, описания, надписи на кнопках.

ЦИКЛЫ И ФОНОВЫЕ ПРОЦЕССЫ

`while(true)` - это бесконечный цикл. Он выполняется снова и снова, пока мы не скажем ему остановиться.

`delay(1000)` внутри цикла заставляет программу ждать 1 секунду перед следующим повторением. Без `delay` цикл работал бы слишком быстро!

`LaunchedEffect(Unit)` запускает код один раз при создании экрана. Это безопасный способ запустить фоновый процесс в Compose.

ФИНАЛЬНАЯ МИССИЯ: КИБЕР-ОХОТНИК 2.0

МОДУЛЬ: СОСТОЯНИЕ, ЦИКЛЫ И ЭКОНОМИКА | ID: 4

ЭТАП 1: 1. Объявляем функцию (Голова)

```
// Любой экран в Compose начинается с аннотации @Composable
@Composable
fun CyberBountyHunterScreen() {
    // ВСЬ КОД ИЗ СЛЕДУЮЩИХ ШАГОВ ПИШЕМ ТУТ, ВНУТРИ ФУНКЦИИ
}
```

ЭТАП 2: 2. Создаем память экрана (Переменные Состояния)

```
// Эти переменные хранят данные. Если они изменятся экран перерисовывается сам.
var credits by remember { mutableStateOf(0L) } // Наши деньги
var clickPower by remember { mutableStateOf(1L) } // Сила одного клика
var drones by remember { mutableStateOf(0L) } // Количество купленных дронов
```

ЭТАП 3: 3. Умная логика Рангов

```
// Переменная rank сама решает, какой текст показать.
val rank = when {
    credits < 100 -> "НОВИЧОК"
    credits < 1000 -> "НАЕМНИК"
    credits < 5000 -> "МАСТЕР"
    else -> "ЛЕГЕНДА ПУСТОШИ"
}
```

ЭТАП 4: 4. Автоматический заработок (LaunchedEffect)

```
// LaunchedEffect создает фоновый процесс.
// while (true) бесконечный цикл, работающий пока открыт экран.
LaunchedEffect(Unit) {
    while (true) {
        delay(1000)
        credits += (drones * 5)
    }
}
```


ФИНАЛЬНАЯ МИССИЯ: КИБЕР-ОХОТНИК 2.0

МОДУЛЬ: СОСТОЯНИЕ, ЦИКЛЫ И ЭКОНОМИКА | ID: 4

ЭТАП 5: 5. Цвета и Главный контейнер (Box)

```
val neonCyan = Color(0xFF00F3FF)
val neonPurple = Color(0xFFBD00FF)

Box(
    modifier = Modifier.fillMaxSize().background(Color(0xFF050505)),
    contentAlignment = Alignment.Center
) {
    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.SpaceBetween
    ) {
        /* ТУТ БУДУТ ТЕКСТЫ И КНОПКИ */
    }
}
```

ЭТАП 6: 6. Верхняя панель: Информация об игроке

```
Column(horizontalAlignment = Alignment.CenterHorizontally) {
    Text("РАНГ: $rank", color = neonPurple, fontWeight = FontWeight.Bold)
    Text("$credits CR", color = neonCyan, fontSize = 50.sp, fontWeight =
FontWeight.Black)
    Row {
        Text("УРОН: $clickPower", color = Color.Gray, fontSize = 12.sp)
        Spacer(Modifier.width(16.dp))
        Text("ДРОНЫ: $drones", color = Color.Gray, fontSize = 12.sp)
    }
}
```


ФИНАЛЬНАЯ МИССИЯ: КИБЕР-ОХОТНИК 2.0

МОДУЛЬ: СОСТОЯНИЕ, ЦИКЛЫ И ЭКОНОМИКА | ID: 4

ЭТАП 7: 7. Главная кнопка: ОХОТА

```
Button(  
    onClick = { credits += clickPower },  
    modifier = Modifier.size(200.dp),  
    shape = CircleShape,  
    colors = ButtonDefaults.buttonColors(containerColor = neonPurple)  
) {  
    Text("ОХОТА", fontSize = 28.sp, fontWeight = FontWeight.Black, color =  
        Color.White)  
}
```

ЭТАП 8: 8. Магазин: Улучшение оружия

```
Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {  
    val weaponCost = clickPower * 15  
    Button(  
        onClick = {  
            if (credits >= weaponCost) {  
                credits -= weaponCost  
                clickPower += 2  
            }  
        },  
        modifier = Modifier.fillMaxWidth().height(50.dp),  
        shape = RoundedCornerShape(8.dp),  
        colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF111111))  
    ) {  
        Text("УЛУЧШИТЬ ПУШКУ (ЦЕНА: $weaponCost)", color = neonCyan)  
    }  
}
```


ФИНАЛЬНАЯ МИССИЯ: КИБЕР-ОХОТНИК 2.0

МОДУЛЬ: СОСТОЯНИЕ, ЦИКЛЫ И ЭКОНОМИКА | ID: 4

ЭТАП 9: 9. Магазин: Покупка Дронов

```
val droneCost = (drones + 1) * 50

Button(
    onClick = {
        if (credits >= droneCost) {
            credits -= droneCost
            drones += 1
        }
    },
    modifier = Modifier.fillMaxWidth().height(50.dp),
    shape = RoundedCornerShape(8.dp),
    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF111111))
) {
    Text("КУПИТЬ ДРОНА (ЦЕНА: $droneCost)", color = neonPurple)
}
```

ЭТАП 10: 10. Создаем Preview

```
// ВАЖНО: Пью пишется СНАРУЖИ основной функции.
@Preview
@Composable
fun BountyHunterPreview() {
    MaterialTheme {
        CyberBountyHunterScreen()
    }
}
```

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Добавь Критический клик: создай переменную Chance. Сделай так, чтобы с шансом 10 процентов (используй Random) клик давал в 10 раз больше денег.